

Assignment 3 | Bridge Pattern

Course: ShP-2216 Software Design Patterns, Astana IT University, 2026-2027 | **Student:** Yershadi Nurassyl | **Group:** SE-2523 | **Topic:** A (Drawing: Shape / Renderer)

Repository: <https://github.com/n4120236-hue/Assignment-3-SDP>

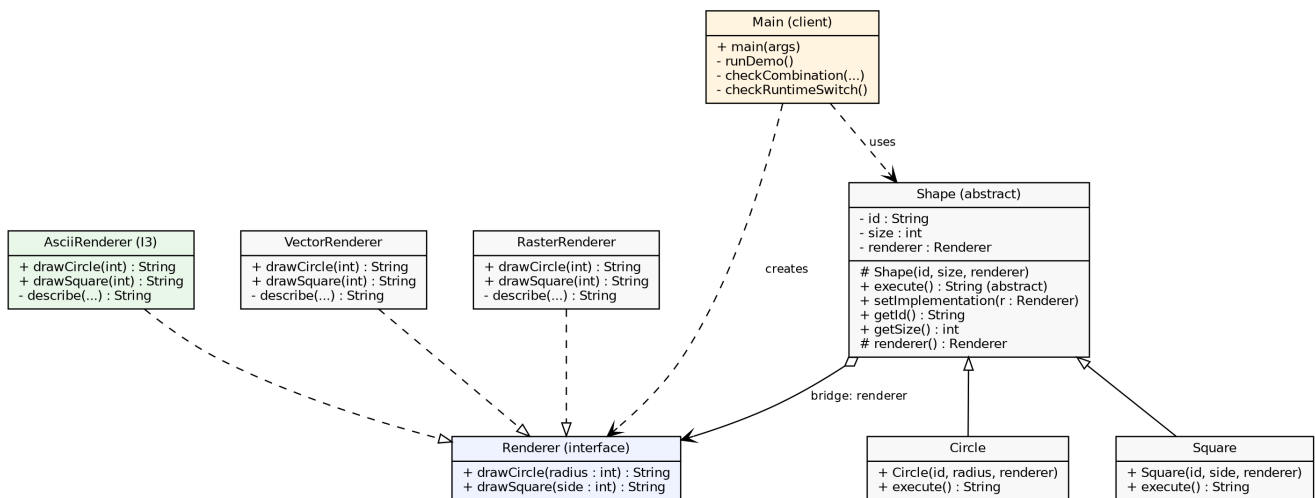
Base commit (I1/I2): e7b1074b829d3e49e10526177411ad3574df3fe5

Submitted commit (I3 extension): b546192142a2f6e4685af9a50af7f0602d618336

1. Two independent dimensions and why Bridge fits

The first dimension is *what is drawn*: the abstraction Shape with refined abstractions Circle (radius 2) and Square (side 3). The second is *how it is drawn*: the implementor interface Renderer with VectorRenderer, RasterRenderer and, after the extension, AsciiRenderer. Without Bridge, every pair would need its own class (CircleVector, CircleRaster, SquareVector, ...), so N shapes and M renderers give N x M classes and each new renderer forces one new class per shape. With Bridge, Shape holds a Renderer reference supplied through the constructor and delegates to it, so the hierarchies grow separately (N + M classes) and the renderer can be replaced at runtime with `setImplementation(...)`.

2. UML class diagram



Shape and Renderer are joined by the field `renderer`, typed with the interface. Circle and Square extend Shape; the three renderers implement Renderer. Main creates the renderers and calls only `Shape.execute()` and `Shape.setImplementation(...)`.

3. Clean Code excerpts

(a) Separated responsibilities - Circle.java

```
@Override
public String execute() {
    return renderer().drawCircle(getSize());
}
```

Circle only decides *which* low-level operation a circle needs. How it looks is decided by the renderer. A new drawing style never touches Circle.

(b) Meaningful names - Renderer.java and VectorRenderer.java

```
String drawCircle(int radius);
String drawSquare(int side);
private String describe(String shape, String dimension, int value)
```

Operations and parameters use domain vocabulary (radius, side, describe), so the code reads without comments and the dimension named in the output matches the parameter.

(c) Small focused methods - Shape.java

```
public void setImplementation(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer);
}
```

The method does one thing: replaces the bridge reference and rejects null. Each renderer method is likewise one delegating line, which makes the runtime switch easy to test.

(d) No duplicated workflow or formatting - VectorRenderer.java and Main.java

```
public String drawCircle(int radius) { return describe("circle", "radius", radius); }
public String drawSquare(int side) { return describe("square", "side", side); }
private static boolean checkCombination(String id, Shape shape, Renderer renderer, String expected)
```

The string layout is written once per renderer in `describe(...)`, and one `checkCombination` method serves T1-T4, T6 and T7, so a format change or a new check touches one place.

(e) Encapsulated state - Shape.java

```
private final String id;  
private final int size;  
private Renderer renderer;  
protected Renderer renderer() { return renderer; }
```

Domain data is private and final, so it cannot change during the runtime switch (T5). The renderer field is private and reachable by subclasses only through a protected accessor, and only setImplementation(...) can replace it.

4. Extension with I3, trade-off and comparison with Adapter

How I3 was added. After committing the working I1/I2 version (base commit above), I added the class AsciiRenderer implementing Renderer, and Main gained T6/T7 and the ascii instance. `git diff BASE HEAD -- src` (saved as extension.diff) shows exactly two files: the new src/bridge/AsciiRenderer.java and src/Main.java. **Unchanged:** Shape, Circle, Square, Renderer, VectorRenderer and RasterRenderer.

Trade-off. Bridge adds an extra interface and one more level of indirection, so a small program with one fixed renderer becomes harder to read. The Renderer contract must also be chosen carefully: adding a new shape later (for example Triangle) would need a new method in Renderer and therefore changes in every renderer.

Bridge vs Adapter. Bridge is a design-time decision: the two hierarchies are planned up front so that they can vary independently, and Shape is written to work with any Renderer. Adapter is applied afterwards to make an existing, incompatible class fit an interface the client already expects (for example wrapping a third-party drawing library so it looks like Renderer). In this application, Shape and Renderer were designed together for independent extension, which is Bridge; an Adapter would only appear if an external library had to be plugged in as one more Renderer.

5. Sources

[1] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 (Bridge, Adapter). [2] Lecture 4, Bridge Pattern, ShP-2216 Software Design Patterns, Astana IT University, 2026-2027, and the Bridge reading on Moodle. [3] R. C. Martin, *Clean Code*, Prentice Hall, 2008. [4] Refactoring.Guru, "Bridge", <https://refactoring.guru/design-patterns/bridge>.